

HDOC Day-23

Write a C program that displays the following menu and uses a switch statement to print the basic square/rectangular pattern selected by the user. Use nested loops without arrays. Each pattern may be implemented in a separate parameterless function with no return value.

Note: For each activity, prepare the algorithm and test-case table. Then write, compile, and execute the C program, and test it against the prepared test cases.

Algorithm: Menu-driven pattern program

1. Start.
2. Display the menu:
 1. Solid square pattern
 2. Solid rectangular pattern
 3. Row-number pattern
 4. Column-number pattern
3. Read the user's choice.
4. Use a switch statement to select the required pattern.
5. For **choice 1**, input n and call the solid-square function.
6. For **choice 2**, input m and n and call the solid-rectangle function.
7. For **choice 3**, input m and n and call the row-number function.
8. For **choice 4**, input m and n and call the column-number function.
9. If the choice is invalid, display an error message.
10. Stop.

C Program:

```
#include <stdio.h>
```

```
/* Function prototypes */

void solidSquare();

void solidRectangle();

void rowNumberPattern();

void columnNumberPattern();


int main()

{

    int choice;


    printf("==== PATTERN MENU =====\n");

    printf("1. Solid Square Pattern\n");

    printf("2. Solid Rectangular Pattern\n");

    printf("3. Row-Number Pattern\n");

    printf("4. Column-Number Pattern\n");

    printf("Enter your choice: ");

    scanf("%d", &choice);


    switch(choice)

    {

        case 1:

            solidSquare();

            break;


        case 2:

            solidRectangle();

            break;
```

case 3:

rowNumberPattern();

break;

case 4:

columnNumberPattern();

break;

default:

printf("Invalid choice!\n");

}

return 0;

}

/* Function for solid square pattern */

void solidSquare()

{

int n, i, j;

printf("Enter the number of rows: ");

scanf("%d", &n);

for(i = 1; i <= n; i++)

{

for(j = 1; j <= n; j++)

{

printf("* ");

```
    }  
    printf("\n");  
}  
}
```

```
/* Function for solid rectangular pattern */
```

```
void solidRectangle()
```

```
{
```

```
    int m, n, i, j;
```

```
    printf("Enter the number of rows: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter the number of columns: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 1; i <= m; i++)
```

```
    {
```

```
        for(j = 1; j <= n; j++)
```

```
        {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
/* Function for row-number pattern */
```

```
void rowNumberPattern()
```

```
{  
    int m, n, i, j;  
  
    printf("Enter the number of rows: ");  
    scanf("%d", &m);  
  
    printf("Enter the number of columns: ");  
    scanf("%d", &n);  
  
    for(i = 1; i <= m; i++)  
    {  
        for(j = 1; j <= n; j++)  
        {  
            printf("%d ", i);  
        }  
        printf("\n");  
    }  
}
```

/* Function for column-number pattern */

```
void columnNumberPattern()
```

```
{  
    int m, n, i, j;  
  
    printf("Enter the number of rows: ");  
    scanf("%d", &m);  
  
    printf("Enter the number of columns: ");
```

```
scanf("%d", &n);
```

```
for(i = 1; i <= m; i++)
```

```
{
```

```
    for(j = 1; j <= n; j++)
```

```
    {
```

```
        printf("%d ", j);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
}
```

Test Case Table:

Test Case	Choice	Input	Expected Output
1	1	n = 4	4 * 4 solid square
2	1	n = 2	2 * 2 solid square
3	2	m = 3, n = 5	3 * 5 solid rectangle
4	2	m = 2, n = 4	2 * 4 solid rectangle
5	3	m = 4, n = 4	Rows contain 1, 2, 3, 4 respectively
6	3	m = 3, n = 5	Each row contains the row number 5 times
7	5	-	Invalid choice

Compilation and Execution:

Test Case 1:

```
(kali㉿kali)-[~]  
$ ./a.out  
———— PATTERN MENU ————  
1. Solid Square Pattern  
2. Solid Rectangular Pattern  
3. Row-Number Pattern  
4. Column-Number Pattern  
Enter your choice: 1  
Enter the number of rows:4  
* * * *  
* * * *  
* * * *  
* * * *
```

Test Case 2:

```
(kali㉿kali)-[~]  
$ ./a.out  
———— PATTERN MENU ————  
1. Solid Square Pattern  
2. Solid Rectangular Pattern  
3. Row-Number Pattern  
4. Column-Number Pattern  
Enter your choice: 1  
Enter the number of rows:2  
* *  
* *
```

Test Case 3:

```
(kali@kali)-[~]  
$ ./a.out  
———— PATTERN MENU ————  
1. Solid Square Pattern  
2. Solid Rectangular Pattern  
3. Row-Number Pattern  
4. Column-Number Pattern  
Enter your choice: 2  
Enter the number of rows: 5  
Enter the number of columns: 3  
* * *  
* * *  
* * *  
* * *  
* * *
```

Test Case 4:

```
(kali@kali)-[~]  
$ ./a.out  
———— PATTERN MENU ————  
1. Solid Square Pattern  
2. Solid Rectangular Pattern  
3. Row-Number Pattern  
4. Column-Number Pattern  
Enter your choice: 2  
Enter the number of rows: 4  
Enter the number of columns: 2  
* *  
* *
```

Test Case 5:

```
(kali㉿kali)-[~]  
$ ./a.out  
----- PATTERN MENU -----  
1. Solid Square Pattern  
2. Solid Rectangular Pattern  
3. Row-Number Pattern  
4. Column-Number Pattern  
Enter your choice: 3  
Enter the number of rows: 4  
Enter the number of columns: 4  
1111  
2222  
3333  
4444
```

Test Case 6:

```
(kali㉿kali)-[~]  
$ ./a.out  
----- PATTERN MENU -----  
1. Solid Square Pattern  
2. Solid Rectangular Pattern  
3. Row-Number Pattern  
4. Column-Number Pattern  
Enter your choice: 3  
Enter the number of rows: 5  
Enter the number of columns: 3  
11111  
22222  
33333  
44444  
55555
```

Test Case 7:

```
(kali㉿kali)-[~]  
$ ./a.out  
----- PATTERN MENU -----  
1. Solid Square Pattern  
2. Solid Rectangular Pattern  
3. Row-Number Pattern  
4. Column-Number Pattern  
Enter your choice: 5  
Invalid choice!
```